Neural Network Techniques for Move Ordering in Chess Engines

Intended Format: Experimental
Group Members: Max Campbell, z5691034, Ning Xiaozhou, z5693291, Warren Huang, z5764242
Mentor: Dr Mashbat Suzuki

Topic and Objective

Alpha-beta pruning computes exactly the minimax value of a game tree, but how much of the tree it examines depends entirely on the order in which moves are searched. If the best move is always tried first, alpha-beta examines roughly $b^{d/2}$ leaves of a tree with branching factor b and depth d, instead of the b^d examined by minimax; with poor ordering it saves little compared to pure minimax (Knuth & Moore, 1975).

This technique of alpha-beta pruning is very applicable to games with deep and complex trees such as chess and can save huge sums of time. For this reason, alpha-beta pruning has wide consensus to be required in top modern day chess computers or engines. However, alpha-beta pruning only describes how to consider what calculations to do further down in the game tree, it still leaves a decision for which order to consider candidate moves at each node.

A field of techniques known as 'move ordering' is the solution to this problem, such as the history heuristic and killer moves heuristic both having been developed in the 70's and 80's (Akl & Newborn, 1977; Schaeffer, 1983). In modern top chess engines a large portion of their strength can be attributed to excellent move ordering, such as Ethereal where ~800 Elo is attributed to quiet move ordering alone (Grant, n.d.). The very best chess engines such as Stockfish achieve an effective branching factor (EBF) near or below 2 with all their pruning enabled (Chess Programming Wiki, n.d.-a).

There have been many attempts over the years to improve the move ordering, but a combination of hand-crafted heuristics like the History Heuristic (Schaeffer, 1983) are still state of the art despite being more than 43 years old. Multiple other areas of Chess Engine programming have undergone revolutions due to implementing Neural Networks, such as in evaluation (Nasu, 2018; Stockfish developers, 2020; Grant, 2020). Policy networks in AlphaZero (Silver et al., 2018) and more recently, Leela Chess Zero have shown excellence in ordering moves accurately, even reaching up to 2000+ Elo on depth 1 search, in other words relying entirely on its policy network (Ruoss et al., 2024).

The question: Will adding a neural-network based move-ordering heuristic to alpha-beta search work in 2026 after all the new progress in other fields of chess engine research?

To answer this question, we need to ask two questions:

- Can a neural network beat the state-of-the-art move ordering heuristics found in top chess engines in reducing EBF and node count?
- Will a neural network of this size be practical to implement in a real search (Especially with NNUE)? In other words, will the reduced speed caused by neural network ordering be offset by the extra pruning that higher quality ordering provides?

Initial Background

Every top alpha-beta chess engine as of 2026 relies on cheap hand-crafted move ordering heuristics, mainly using a combination of the History Heuristic, Continuation History heuristic (which has replaced the roles of Killers and Countermoves as a more generalised version in the last 10 years), and Capture History. Capture History introduced by Stefan Geschwentner in 2016 was the last significant move ordering innovation to gain any Elo in top-level play (Chess Programming Wiki, n.d.-b). A neural network can predict good moves far more accurately, but each prediction costs more CPU time.

However, with recent advances in chess programming, it is now possible to efficiently update the neural network's activations during gameplay (Stockfish NNUE), using incremental updates rather than fully re-evaluating the network each node (Nasu, 2018; Stockfish developers, 2020). We believe there is nothing stopping this innovation on the evaluation side of chess engines from being applied to policy networks used for move ordering too.

A neural network move ordering applied to chess has been attempted before in the form of the Neural MoveMap heuristic (Kocsis et al., 2003), but at that point in time only one type of neural network architecture was attempted, and further advances like NNUE were not discovered yet, making their network inefficient since it needs to re-evaluate the full network on each node. Additionally, modern heuristics like capture history and continuation history were not in the state-of-the-art engines at the time, and even history heuristic was not using the modern history gravity formula (Chess Programming Wiki, n.d.-b).

Adding a neural move-ordering heuristic to order moves should reduce the number of nodes needed to reach a fixed depth when searching positions, since a network that aims to predict the best move will cause cutoffs earlier more often than simple rules, although the saving may not cover the network's cost, and will only modestly reduce node count against a tuned engine. A standard history heuristic requires just a few table lookups per move compared to a neural network which must be evaluated at every node where moves are ordered, where the majority of these nodes are near the leaves of the tree, meaning that even a perfect ordering can only save in a small number of positions. However, the neural network is extremely beneficial close to the root of the tree, where each individual cutoff can remove a significant number of positions, and has potential when the size of the network is small enough so that evaluation adds close to nothing to the cost of each node throughout the tree.

Proposed approaches and evaluation

We will train a model that uses only the current chess position to predict a probability for each legal move being Stockfish 19's best move (Stockfish developers, 2026). The model is trained with log loss. During alpha-beta search, moves are searched from highest to lowest predicted probability. We compare this with the history heuristic and the other hand-crafted heuristics listed below using runtime, accuracy, and effective branching factor (EBF), which serves as a measure of search complexity.

This is reasonable because alpha-beta prunes best when cutoff moves are searched early. Stockfish 19's best move gives a good target for move ordering, as even on low depth it is close to an oracle. Predicting a full distribution is also necessary, since if the first move does not cause a cutoff, the order of the remaining moves still affects pruning. Log loss rewards good probability estimates across all legal moves.

The input contains 12 piece-type × 8 × 8 occupancy grids, giving 768 binary values, plus side to move, castling rights, and the en passant square. The output gives one score for each from-to square pair, masks illegal moves, and applies a softmax over legal moves.

Training positions will come from Lichess games (Lichess, n.d.) and top chess engine games. Each will be labelled with Stockfish 19's best move using a fixed node budget on one thread, making labels deterministic. We will start with a small feed-forward neural network (similar to Stockfish NNUE) and may also test CNNs or transformers (more similar to Lc0's policy layer).

Methodology

We will test on several hundred held-out opening, middlegame, and endgame positions at a practical depth, perhaps 7 or 8.

On each position we will measure the EBF of common strategies as our control group, and of our neural strategies. This includes, but may not be limited to:

- History Heuristic
- History + Continuation History (current state of the art for quiet move ordering)
- Neural strategy
- Neural strategy + Continuation History (neural strategy acts as an alternative to history)

Note that these are all quiet-move specific ordering heuristics. It is well known that move ordering heuristics based on positional knowledge (such as Neural Networks) are less effective in tactical positions. Therefore if we add any noisy-move heuristics (e.g. for captures), we will add them to all strategies.

Testing will be done with all forward pruning and selective reductions disabled, as these affect the tree topology and EBF which we are trying to optimize.

Prediction quality of the neural network is measured using log loss and top-1 agreement with Stockfish 19.

Note: EBF is defined as EBF = √(N(d) / N(d−2)) (Chess Programming Wiki, n.d.-a), where N(d) is the number of nodes searched to depth d. Since node count grows roughly as EBF^d, lower EBF means more effective pruning.

Consistency

For correctness, every move-ordering method must return the same root value at each depth. Since move ordering cannot change the minimax result, any difference from pure alpha-beta indicates a bug.

All methods use the same code, search, evaluation function, move generator, search depths, positions, and hardware; only move ordering changes. Data is split by game, so no test game appears in training or tuning.

Risks

The main risk is that model inference costs more time than it saves through pruning. This is still fine, as a negative result here is still noteworthy. Node counts, EBF, and cutoff rates still show whether move ordering improved. We can then measure the inference cost to estimate the break-even point. If needed, we can use a smaller model.

Feasibility and Collaboration

Week	Milestone and concrete outcome
3–4	Create and share the repository with group members; Create final report shared document
5	Finish generating/sourcing and labelling test data; Write a test harness able to benchmark ordering strategy quality of different strategies
6	Train the neural network; Measure the log loss of the network, based on agreement with Stockfish; Implement all common move ordering strategies from the basic to the state of the art in our move ordering quality benchmark harness
7 (progression review)	Add the neural network move ordering to the move ordering quality benchmarks; Begin reimplementing the neural network as a sort of incrementally updated NNUE network
8	Investigate different combinations of neural network and ordering heuristics, and find the move ordering quality out of all of them; Profile the performance of our custom ‘NNUE’ implementation and begin optimising it
9	Run the final benchmark of the best move ordering quality found; Run performance benchmarks to measure NNUE; (Optional extension) Run an SPRT Elo test between NNUE and classical ordering
10	Produce tables/plots for node count, EBF, cutoff rates, accuracy, and performance; Clean the repository, document reproducibility instructions; Prepare the poster and final report

Collaboration

The team meets in person on campus once a week to check progress against the milestones, agree on the tasks for the following week and reassign work where someone is blocked, with a short record of decisions and action items kept in either a shared Google Doc or our Discord groupchat. All code is kept in a shared GitHub repository, where each member works on a separate branch and changes are merged into the main branch. Individual contributions are recorded through the commit history, and the review comments.

References

Akl, S. G., & Newborn, M. M. (1977). The principal continuation and the killer heuristic. *Proceedings of the 1977 ACM Annual Conference*, 466–473.

Chess Programming Wiki. (n.d.-a). Branching factor. https://www.chessprogramming.org/Branching_Factor

Chess Programming Wiki. (n.d.-b). History heuristic. https://www.chessprogramming.org/History_Heuristic

Grant, A. (n.d.). *Ethereal* [Computer software]. GitHub. https://github.com/AndyGrant/Ethereal

Grant, A. (2020). Evaluation & tuning in chess engines. TalkChess forum. https://talkchess.com/viewtopic.php?t=74877

Knuth, D. E., & Moore, R. W. (1975). An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4), 293–326.

Kocsis, L., Uiterwijk, J. W. H. M., Postma, E., & van den Herik, H. J. (2003). The Neural MoveMap heuristic in chess. In *Computers and Games (CG 2002)*, LNCS 2883. Springer. https://doi.org/10.1007/978-3-540-40031-8_11

Lichess. (n.d.). *Lichess open database*. https://database.lichess.org

Nasu, Y. (2018). *Efficiently updatable neural-network-based evaluation functions for computer shogi* [in Japanese]. Ziosoft Computer Shogi Club.

Ruoss, A., et al. (2024). Amortized planning with large-scale transformers: A case study on chess. *Advances in Neural Information Processing Systems 37*. https://arxiv.org/abs/2402.04494

Schaeffer, J. (1983). The history heuristic. *ICCA Journal*, 6(3). https://doi.org/10.3233/ICG-1983-6305

Silver, D., et al. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419), 1140–1144.

Stockfish developers. (2020, August 7). *Introducing NNUE evaluation*. https://stockfishchess.org/blog/2020/introducing-nnue-evaluation/

Stockfish developers. (2026, September 5). *Stockfish 19*. https://stockfishchess.org/blog/2026/stockfish-19/

AI Statement

AI was used for formatting and referencing purposes.